**University of Warsaw**
Faculty of Mathematics, Informatics and Mechanics

**Pavel Charheika**
Student no. 451309

**Franciszek Skawiński**
Student no. 439596

**Yevhenii Tarasiuk**
Student no. 459775

**Bogdan Vysocki**
Student no. 466466

# Design and implementation of a customer self-service web portal for a telecommunications operator

**Bachelor's thesis**
**in COMPUTER SCIENCE**

Supervisor:
**Michał Możdżonek**
Instytut Informatyki

Warsaw, June 2026

## Abstract

This thesis presents the design and implementation of a self-service web portal for customers of a telecommunications operator. The project addresses the problem of repetitive customer service interactions that are commonly handled through call centres, physical offices, or manual back-office procedures. These interactions include service management, billing enquiries, payments, refunds, notifications, help materials, and account settings. The proposed portal is intended to centralize routine customer operations and reduce the dependence on direct support channels.

The system is implemented as a web application with a separated client-server architecture. The frontend is developed using Angular and follows a component-service structure with routing, access control, HTTP communication, state handling, and compile-time internationalization. The backend is implemented as a Spring Boot service written in Kotlin and exposes a REST API responsible for authentication, authorization, business logic, validation, data access, caching, and e-mail notifications. The data tier models the main entities required by the portal, including users, services, offers, billing history, messages, and notifications.

The thesis describes the business context and value proposition of the system, its high-level architecture, frontend and backend implementation, database design, API structure, testing approach, and installation procedure. The resulting solution supports customer relationship management by giving users faster access to service and billing information, while helping smaller Internet service providers and local network operators make routine customer service processes more scalable, consistent, and efficient.



## Keywords

telecommunications, self-service portal, customer relationship management, web application, Angular, Spring Boot, Kotlin, REST API, database design, system testing

## Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatyka

## Subject classification

Information systems---World Wide Web---web applications


## Tytuł pracy w języku polskim

Projektowanie i implementacja internetowego biura obsługi klienta dla operatora telekomunikacyjnego

# Contents

**Introduction** . . . . . 5

**1. Project objectives and context** . . . . . 7
1.1. Problem statement and business domain . . . . . 7
1.2. Proposed solution and value proposition . . . . . 7

**2. System architecture** . . . . . 9
2.1. High-level architectural overview . . . . . 9
2.2. Client-side architecture . . . . . 9
2.2.1. Framework choice justification . . . . . 9
2.2.2. Components and Services . . . . . 10
2.2.3. Compile-time internationalization . . . . . 10
2.3. Data tier architecture . . . . . 10
2.4. Server-side architecture . . . . . 11
2.5. Cloud infrastructure and deployment . . . . . 12

**3. Client-side implementation (Frontend)** . . . . . 13
3.1. Client-side technology stack . . . . . 13
3.1.1. The core framework . . . . . 13
3.1.2. Supplementary UI component library . . . . . 13
3.1.3. Client-Server decoupling strategies . . . . . 13
3.2. Structural design and code components . . . . . 13
3.2.1. Directory layout . . . . . 14
3.2.2. Component-Service architecture . . . . . 14
3.2.3. State management and reactivity . . . . . 15
3.2.4. TypeScript, Data Transfer Objects and Interfaces . . . . . 15
3.3. Navigation and routing strategy . . . . . 16
3.3.1. Route hierarchy and layout encapsulation . . . . . 16
3.3.2. Access control via route guards . . . . . 17
3.4. Client-server communication . . . . . 17
3.4.1. The middleware interceptor pipeline . . . . . 17
3.4.2. Binary data stream processing . . . . . 17
3.5. User interface and experience design . . . . . 18
3.5.1. Translating static mockups to interactive layouts . . . . . 18
3.5.2. Iterative design evolution and communication . . . . . 19
3.6. Internationalization and Localization . . . . . 19
3.6.1. Compile-time translation implementation . . . . . 19
3.6.2. Infrastructure routing and request dispatching . . . . . 20

3.6.3. Client-server language synchronization . . . 20

**4. Database design and data modeling** . . . 21
4.1. User management entity . . . 21
4.2. Services and offers catalog . . . 22
4.3. Billing history . . . 22
4.4. Message and notification history . . . 23

**5. Server-side implementation (Backend)** . . . 25
5.1. Technology stack and framework selection . . . 25
5.2. Architectural patterns and core modules . . . 25
5.3. API design and endpoints . . . 26
5.4. Data access layer . . . 29
5.5. Integration with presentation layer . . . 29
5.6. Interactive API documentation . . . 29

**6. System testing and quality assurance** . . . 31
6.1. Unit testing . . . 31
6.2. Integration testing . . . 33
6.3. Functional testing . . . 34
6.3.1. Environment and prerequisites . . . 34
6.3.2. Functional test matrix . . . 35
6.3.3. End-to-End user journey case study: transfer details path . . . 37

**7. Project management and task allocation** . . . 39
7.1. Division of work . . . 39

**8. Conclusion** . . . 45
8.1. Summary of project outcomes . . . 45
8.2. Future enhancements . . . 45

**A. Installation instructions** . . . 47
A.1. System prerequisites . . . 47
A.2. Frontend application initialization . . . 47
A.3. Backend and database initialization . . . 48
A.4. Database state reset . . . 48
A.5. Environment termination . . . 48

**Bibliography** . . . 49

# Introduction

Digital self-service systems have become an important part of modern customer service. They allow users to access information and perform routine operations independently, while reducing the workload of support staff. In the telecommunications domain, this is particularly relevant because customers regularly interact with the operator in matters related to services, payments, technical information, and account management.

This thesis focuses on the development of a customer-facing web portal for a telecommunications operator. The main motivation for the project is to provide a practical solution that improves the availability of routine customer operations and supports more efficient communication between the operator and its users. The system is designed as a web application with a separated frontend, backend, and data tier.

The work presents both the business and technical aspects of the project. It starts with the project objectives and business context, then describes the system architecture, frontend implementation, database model, backend implementation, and testing approach. The final part summarizes the achieved results and outlines possible directions for further development.

# Chapter 1

# Project objectives and context

## 1.1. Problem statement and business domain

The telecommunications industry is characterised by a high volume of recurring customer interactions, including service activation and management, billing enquiries, technical support requests, and personal data updates. Traditionally, many of these operations are handled through call centres, physical retail offices, or manual back-office procedures. While such channels remain important, they often involve considerable operational costs, limited scalability, and long response times.

These issues are directly related to customer relationship management, as the effective handling of service interactions affects both service quality and user satisfaction. The problem is especially relevant for smaller Internet service providers and local network operators, which may have limited resources for developing advanced support systems. Therefore, there is a need for a self-service solution that can reduce dependence on support channels and make routine customer service operations more scalable and efficient.

## 1.2. Proposed solution and value proposition

A self-service web portal is proposed as a digital access point between a telecommunications operator and its customers. It provides access to core functions, including service management, billing information, payments, refunds, help materials, notifications, and account settings. Through a single interface, customers can manage routine matters, monitor financial information, receive important updates, and find answers to common questions without contacting support staff for every request.

The use of such a portal can reduce the number of repetitive enquiries handled through direct support channels and improve the availability of basic operations. It can also support communication between the operator and its customers by providing information about planned maintenance, technical issues, and relevant offers. As a result, the system can make customer service processes more scalable and efficient, while giving users faster access to information and greater control over their services.

# Chapter 2

# System architecture

## 2.1. High-level architectural overview

At the highest level, the application consists of a frontend web application, a backend service, and a relational database.

The frontend is responsible for presenting the user interface, handling navigation, collecting user input, and communicating with the backend through HTTP requests. It does not access persistent data directly and does not implement server-side business rules.

The backend acts as the central coordination layer of the system. It exposes a REST API consumed by the frontend application and is responsible for request processing, authentication, authorization, validation, business logic, and communication with the database. This design ensures that all operations affecting application data are handled consistently through a controlled server-side interface.

The data tier stores the main domain entities required by the portal, including users, services, offers, billing history, messages, and notifications. Access to this layer is performed only through the backend service, which prevents direct client-side manipulation of persistent data and helps maintain data integrity.

This architecture supports a modular implementation of the portal. The frontend focuses on user interaction, the backend provides application functionality and integration logic, and the database is responsible for durable storage. As a result, the system provides a scalable foundation for customer self-service operations while remaining suitable for further extension and maintenance.

## 2.2. Client-side architecture

### 2.2.1. Framework choice justification

The Angular v21 framework was chosen to implement the frontend application. The choice was primarily driven by the need for technological alignment with the existing technology stack of the industry partner.

Moreover, Angular provides a comprehensive, integrated ecosystem with essential features—such as routing, HTTP communication, and form validation—included out-of-the-box. This native feature set eliminated the overhead of evaluating and integrating third-party libraries, ensuring a highly standardized and cohesive architecture.

Additionally, Angular's native enforcement of TypeScript introduces compile-time static typing, unlike standard JavaScript. By utilizing data types and interfaces, the application

establishes data contracts that mirror the backend API payloads. The type safety minimizes runtime execution errors, catches data mismatches during the build phase, and provides self-documenting code that improves long-term project maintainability.

Furthermore, Angular was chosen because it includes native internationalization capabilities directly within its core ecosystem. This eliminated the need to rely on third-party runtime translation libraries, allowing the project to leverage compile-time localization optimizations.

### 2.2.2. Components and Services

The application implements a strict separation of concerns, dividing code into presentation components and data-fetching services. This industry-standard architecture decouples the frontend-only logic and UI from communication with backend, which facilitates testing and maintenance.

Component responsibilities are limited to rendering data, managing local component state, and capturing user input events.

Services, on the other hand, act as the bridge between API and user interface by handling HTTP requests.

### 2.2.3. Compile-time internationalization

To support deployment across multiple linguistic regions, the application implements a localization architecture.

Traditional client-side localization relies on runtime interpolation, where JavaScript libraries intercept text strings in browser memory and swap them dynamically. This approach can cause layout shifting during page loads and limits search engine indexability.

The integration uses the core Angular internationalization framework (`@angular/localize`) and its compile-time localization matching single-language performance. This avoids the runtime performance penalties associated with third-party translation libraries.

## 2.3. Data tier architecture

The data tier serves as the persistent storage foundation of the system, responsible for securely managing all application state, user profiles, financial ledgers, and service catalogs. It is implemented using PostgreSQL 17, a robust open-source relational database management system chosen for its strong adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties and advanced data-type support.

To address the varied structural requirements of the telecommunications domain, the architecture employs a hybrid data modeling strategy. Core business entities and their relationships are modeled using strict relational paradigms, utilizing explicit foreign keys, associative entities, and database-level unique constraints to enforce referential integrity. However, for highly dynamic features—such as customizable UI components within marketing offers or variable payloads in targeted user notifications—the schema leverages PostgreSQL's native `JSONB` and `TEXT` representations of JSON data. This hybrid approach allows the system to store unstructured, schemaless data efficiently while maintaining the structural rigor of a relational database for critical business transactions.

Database schema evolution is strictly controlled and versioned using Liquibase. Rather than relying on the Object-Relational Mapping (ORM) framework to automatically generate or alter database tables at runtime, all structural changes are defined as explicit, deterministic

changelogs. This methodology guarantees that database migrations are predictable, reproducible across different development and production environments, and safely integrated into the application's version control system.

From a security and infrastructure perspective, the database operates within an isolated Docker container hosted on the designated AWS environment. It is intentionally shielded from the public internet, restricting direct inbound network connections. All data access, retrieval, and manipulation are routed exclusively through the backend's Spring Data JPA layer via the Hibernate ORM. This architectural constraint ensures that all database interactions are subjected to the backend's authorization checks and business logic validations, thereby establishing a single, secure point of entry that prevents unauthorized data access and maintains overall data integrity.

## 2.4. Server-side architecture

The server-side architecture is built upon the Spring Boot framework using Kotlin, exposing a stateless RESTful API to the client application. This backend serves as the central coordination layer of the system, mediating all interactions between the client-side presentation and the underlying PostgreSQL data tier. By relying on a stateless design, the server does not maintain client sessions in memory, which facilitates predictable execution and reduces server-side overhead.

To enforce strict separation of concerns and maintainability, the backend is organized into distinct, isolated architectural modules:

- **Presentation (API) Layer:** Exposes the REST controllers, manages request routing, handles Jakarta Bean Validation, and centralizes exception handling. It ensures that the public contract is expressed exclusively through Data Transfer Objects (DTOs), decoupling external representations from the internal data model.

- **Service Layer:** Encapsulates the core business rules of the application. It is responsible for orchestrating transactional operations, computing billing balances, managing state transitions, and translating between persistent entities and DTOs.

- **Infrastructure Layer:** Abstracts all database interactions, providing Spring Data JPA repositories and managing the Liquibase database migration configurations.

- **Domain Layer:** Represents the core of the system, containing the pure JPA-mapped persistent entities (e.g., `Client`, `Invoice`, `Offer`) without any dependencies on outer architectural layers.

The security architecture utilizes a stateless authentication pipeline managed by Spring Security. Initial system access is granted via a One-Time Password (OTP) verification flow. Upon successful verification, a JSON Web Token (JWT) is issued to the client, which is subsequently used as a bearer token to authorize all ensuing API requests securely.

Beyond standard data processing, the server-side architecture integrates specialized cross-cutting components. This includes the Spring Cache module for optimizing read operations, Spring Mail for dispatching automated outbound notifications, and a dedicated document engine that generates binary PDF artifacts (such as invoices and legal documents) directly from relational records using the OpenPDF library.

## 2.5. Cloud infrastructure and deployment

The deployment environment was provided by the industry partner as a server hosted on Amazon Web Services (AWS). The server contains separate directories for the backend application and the frontend web files. The backend is located in `/apps/be/dev/`, while the frontend application served by Nginx is located in `/apps/www/bok.app`.

The infrastructure uses Docker Compose to run the database and the Nginx web server. The Docker Compose configuration is stored in `/apps/docker-compose.yml`. The database is available only inside the server environment and is not exposed directly to the public network. External access to the database requires an SSH tunnel, while connection parameters are stored in an environment file on the server.

The backend application is started directly on the server as a Java process, without Docker. A startup script is used to run the application with the development Spring profile and an external configuration file. The backend listens on a local port and is accessed by other server-side components rather than directly from the public Internet.

# Chapter 3

# Client-side implementation (Frontend)

## 3.1. Client-side technology stack

### 3.1.1. The core framework

As mentioned in Subsection 2.2.1, the Angular v21 framework (version 21.1) was chosen to align with the technology stack of the partner company, leverage TypeScript's compile-time type verification, and gain access to the integrated ecosystem's features, such as routing, HTTP communication, and form validation.

### 3.1.2. Supplementary UI component library

Although the foundational presentation layer was developed using custom HTML and CSS, the PrimeNG component library (version 21.1) [6] was integrated to handle complex, interactive UI elements. This approach provided access to pre-built, thoroughly tested, and accessible components, thereby accelerating development velocity and ensuring a consistent user experience.

### 3.1.3. Client-Server decoupling strategies

To mitigate the inherent synchronization bottlenecks between client-side and server-side engineering, the project utilized Prism (version 5.14.2) [5] for strictly defined API simulation tooling. By defining expected data structures and endpoint behaviors early in the development cycle—whether through formal OpenAPI (version 3.0.0) specifications or iterative, high-level structural descriptions—the frontend team was able to eliminate backend synchronization delays. Consequently, this approach successfully decoupled frontend development from the backend implementation state.

## 3.2. Structural design and code components

The client-side application is built upon a modular, component-service architecture designed to enforce a strict separation of concerns and maximize code maintainability. This section outlines the directory structure of the frontend codebase, details the interaction patterns between the presentation layer and data services, and examines the reactive state management strategy utilized.

### 3.2.1. Directory layout

The application uses a feature-driven directory layout to separate configuration files, core routing, and feature-specific business logic.

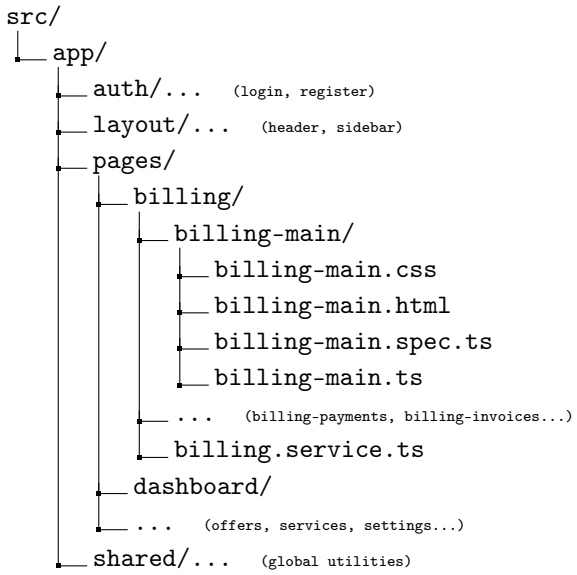



Figure 3.1: Truncated application directory layout demonstrating feature-driven organization.

The root workspace contains configuration files, deployment and development tools, as well as the `mocks` directory used for the Prism API simulation. The compiled build output is stored in the `dist` folder.

The `src` directory contains the application code, alongside static `assets`, environment variables, and the `locale` directory for internationalization configurations.

The core architecture is defined within the `src/app` directory, which is divided into the following primary modules:

- `auth`: Isolates user authentication flow;
- `layout`: Contains structural interface components, specifically the `header`, `sidebar`, and `layout-main` components, along with their state service;
- `pages`: Implements the feature-based organization strategy. It groups UI components and services by business domain into subdirectories such as `billing`, `dashboard`, `help`, `messages`, `offers`, `services`, and `settings`;
- `shared`: Stores common UI components and generic services required by multiple page modules;

### 3.2.2. Component-Service architecture

As stated in subsection 2.2.3, the application separates presentation components from data-fetching services. Services encapsulate business logic and data management. They act as the primary interface between the frontend and the backend API. By injecting services into components, the application isolates client-server communication from the business logic and the presentation layer, improving code maintainability.

#### Example

Listing 3.1 illustrates the presentation layer implementation. The component relies on a reactive Angular Signal to store state, but delegates the data retrieval responsibility to the injected service during the initialization lifecycle hook (`ngOnInit()`).

```
async loadSettings(): Promise<void> {
    this.ready.set(false);

    try {
        const response = await firstValueFrom(
            this.settingsService.getNotificationSettings()
        );

        this.applyResponse(response);
        this.ready.set(true);
    } catch (err) {
        // ...
    }
}
```

Listing 3.1: Presentation layer lifecycle hook utilizing an Angular Signal.

Listing 3.2 isolates the network communication mechanics. The service handles the HTTP GET request, targets the configured environment API endpoint, and passes standard configuration options, returning an asynchronous stream back to the UI controller.

```
getNotificationSettings(): Observable<NotificationSettingsResponse> {
    return this.http.get<NotificationSettingsResponse>(
        `${this.apiUrl}/clients/lang-notification-settings`,
        this.getStandardOptions()
    );
}
```

Listing 3.2: Data service encapsulating the HTTP network communication logic.

This structure allows developers to modify API endpoints or data processing rules within the service without requiring changes to the corresponding UI components.

### 3.2.3. State management and reactivity

The application manages client-side state dynamically to sync the user interface with asynchronous backend responses. The presentation layer uses a reactive architecture built on Angular Signals [2].

A signal represents a wrapper around a value that notifies consumer components whenever that value changes. This mechanism allows the framework to execute small-sized updates, refreshing only the specific UI nodes tied to the modified data.

The application utilizes signals to handle data states, loading indicators, and user interactions. When an HTTP response arrives from the server, the component updates the signal value. The framework automatically detects the change and updates the target template.

### 3.2.4. TypeScript, Data Transfer Objects and Interfaces

To ensure data consistency, the application maps backend communication schemas to client-side types and interfaces. These models mirror the payloads transmitted by the backend.

While certain data streams utilize the `any` type keyword to maintain development speed and bypass temporary schema shifts, the core data-handling logic enforces strict TypeScript interfaces, such as `NotificationSettingDto`, as demonstrated by Listing 3.3.

```typescript
export interface NotificationSettingDto {
  notificationTypeId: number;
  notificationTypeCode: string;
  notificationTypeName: string;
  channel: string;
  enabled: boolean;
}
```

Listing 3.3: An example of a Data Transfer Object used in the application.

The introduction of these explicit structures provides several advantages, such as reducing runtime crashes, establishing a clear target structure for mapping routines, code readability and maintainability.

## 3.3. Navigation and routing strategy

The app functions as a Single Page Application, utilizing the Angular Router [1] to handle view transitions dynamically while also managing URL changes. The routing architecture isolates public landing zones from secure views wrapped within a master layout component.

### 3.3.1. Route hierarchy and layout encapsulation

The configuration splits the application into two distinct environments based on authentication requirements. Public routes (e.g., the landing screen (`start`) and the authentication workflows (`auth`)) are placed at the root level of the configuration tree. Unauthenticated users can access the registration or login portals without triggering permission checks.

All internal dashboard features are nested as child routes under an empty path bound to the `LayoutMain` component, which serves as a shell, rendering persistent interface elements like the header and the navigation sidebar. When a user navigates to an internal route (e.g., `dashboard` or `billing/paid`), the router injects the target feature component dynamically into the layout's primary viewport while the layout itself stays in place.

```typescript
{
  path: '',
  component: LayoutMain,
  canActivate: [authGuard],
  canActivateChild: [authGuard],
  children: [
    { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
    { path: 'dashboard', component: DashboardMain },
    {
      path: 'billing',
      component: BillingMain,
      children: [
        {
          path: 'paid',
          component: BillingInvoices,
          data: { statuses: [ 'paid', 'pending' ] }
        }
        // unpaid invoices, returns, ...
      ]
    }
    // offers, services, ...
  ]
}
```

Listing 3.4: Truncated routes from `app.routes.ts` with access control via `authGuard`.

### 3.3.2. Access control via route guards

Security boundaries are enforced programmatically using the `authGuard` function. By attaching the `canActivate` directive to the parent layout route, the application blocks unauthorized access to the entire internal tree from a single declaration point.

The guard synchronously checks client-side tokens or session flags before allowing the navigation cycle to complete. If the verification fails, the guard cancels the routing operation and redirects the user back to the `start` page. This architecture guarantees that protected data is never shown unless valid credentials are confirmed.

## 3.4. Client-server communication

The application communicates with the backend via asynchronous HTTP requests. The client-server network architecture is centralized using Angular's native `HttpClient` library [3], which handles data streams using RxJS Observables and `async/await` promises.

### 3.4.1. The middleware interceptor pipeline

To enforce cross-cutting concerns globally without modifying individual service files, the application implements an HTTP interceptor pipeline. Before the proxy forwards client requests to the backend architecture, the Express.js application context utilizes native middleware (`express.json()` and `express.urlencoded()`) to parse incoming JSON payloads and URL-encoded query parameters, ensuring the data is strictly formatted before Axios initiates the secondary transmission. Every outgoing request and incoming response passes through this middleware layer.

- **Language synchronization:** The pipeline dynamically inspects the active localized URL context (e.g., `/pl/` or `/en/`) and automatically appends the corresponding locale to the `Accept-Language` request header. This guarantees that the backend serves localized error messages or data payloads corresponding with the user's interface language.
- **Automated token verification:** For secure endpoints, the interceptor intercepts the request and injects authorization headers, such as bearer tokens retrieved from client-side `localStorage`. Then, the response interceptor serves as a global security guard. If the backend returns an HTTP `401 Unauthorized` response due to an expired or missing token, the interceptor halts the application flow, clears local session data, and forces a redirect back to the public route `/start`.

### 3.4.2. Binary data stream processing

While standard JSON payloads are parsed automatically by the framework, retrieving generated artifacts (e.g., invoice PDFs) requires explicit low-level byte management. When requesting a binary file, standard text parsing must be overridden by specifying a binary-safe configuration:

```
getInvoicePdf(invoiceId: number) : Observable<Blob> {
  return this.http.get(`${this.apiUrl}/invoices/${invoiceId}/download`, {
    responseType: 'blob'
  });
}
```

Listing 3.5: Example of a method acquiring binary data specifying `responseType:'blob'`.

Setting the `responseType` parameter to `'blob'` instructs the underlying browser engine to process the raw binary stream into a localized `Blob` object rather than interpreting the incoming bytes as a UTF-8 string.

However, because production traffic passes through a local Node.js reverse proxy (`prod-server.js`), the middleware layer must also account for this binary format. By default, standard proxy HTTP utilities interpret downstream data packages as text, which permanently scrambles raw PDF formatting. To preserve binary integrity, the proxy server explicitly configures its Axios transmission parameters with `responseType: 'arraybuffer'`.

Once received by the browser, the component safely converts this stable memory buffer into a temporary Document Object Model (DOM) link using `URL.createObjectURL(blob)`. The application generates a virtual anchor element programmatically, assigns the object URL, triggers a simulated user click event to prompt the browser's native file download dialog, and immediately revokes the object URL to prevent local memory leaks.

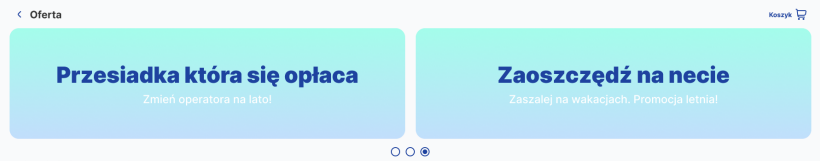
## 3.5. User interface and experience design

The interface design process followed an iterative design-to-code methodology, aligning the technical execution with the visual identity of the company by regular communication.

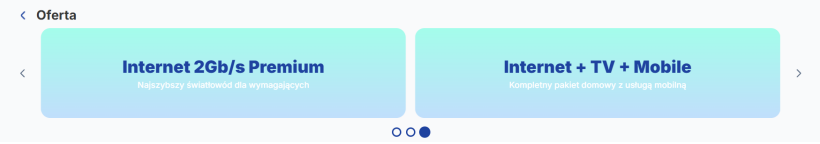
### 3.5.1. Translating static mockups to interactive layouts

The initial design requirements were provided by the partner company in the form of high-fidelity static mockups engineered in Figma. These designs established the core layout grids, color palettes, typography hierarchies, and component structures. The development team translated these visual layouts into HTML templates and CSS styles.

To maintain presentation consistency without creating excessive styling overhead, some complex elements were mapped directly to components from the PrimeNG library (e.g., `p-carousel` and `p-select`). These pre-built modules were styled using custom CSS overrides to match the target design specifications provided by the partner organization with minimal differences.




(a) Initial Figma mockup design.



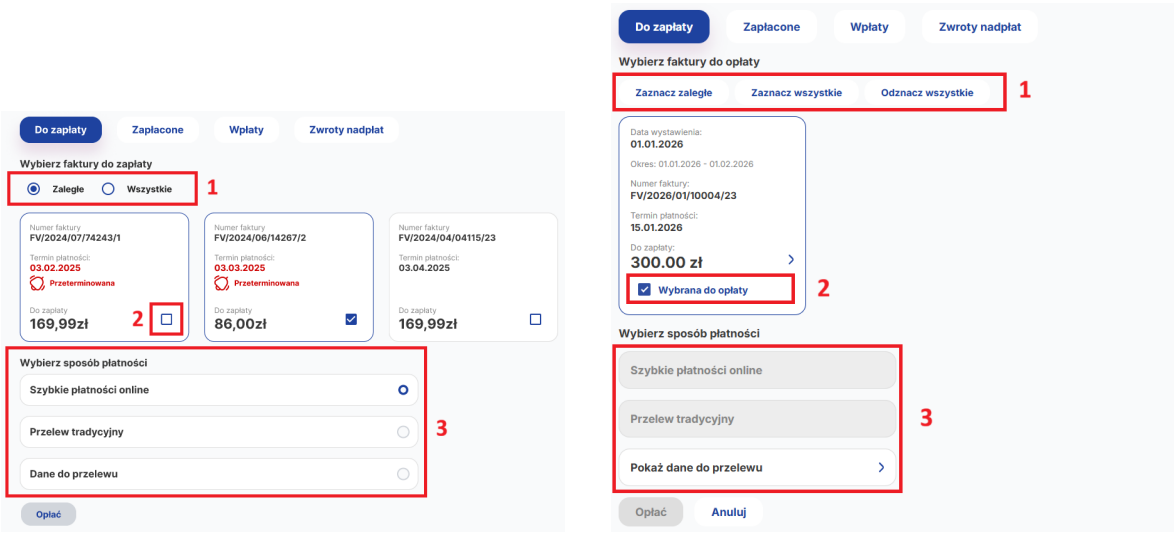

(b) Implemented layout using the `p-carousel` element.

Figure 3.2: Visual comparison of the user interface between the initial design and the deployed app.

### 3.5.2. Iterative design evolution and communication

Throughout the development lifecycle, regular review sessions with the client organization led to iterative refinements of the application layout. Technical limitations discovered during API integration and edge cases in responsive browser behavior necessitated minor layout modifications.

For instance, the payment menu was changed after the parent company decided to scale down the initial project scope, removing two automated payment methods in favor of a simplified transaction workflow. Concurrently, the selection filters were restructured to improve logical grouping and clarity based on user navigation feedback.




(a) Initial Figma mockup design: initial selection filter designs (1), selection checkboxes (2), and multiple payment methods (3).

(b) Implemented layout: improved filters (1), labeled checkboxes for clarity (2) and reduced payment feature scope (3).

Figure 3.3: Visual comparative analysis of the user interface differences between the initial design specification and the deployed app.

## 3.6. Internationalization and Localization

The application materializes the multi-language requirements by leveraging the core Angular internationalization framework (`@angular/localize`) [4], translating the source configuration into standalone physical assets at build time.

### 3.6.1. Compile-time translation implementation

During the production compilation phase, the Angular compiler parses the source templates, extracts structural attributes marked with the `i18n` flag, and cross-references them with the XML Localization Interchange File Format (`.xlf`) translation manifests stored in the `src/locale/` directory. The build engine outputs fully independent, hardcoded deployment bundles for each target locale:

```
1 dist/elcaro-bok-app/browser/en/
2 dist/elcaro-bok-app/browser/pl/
```

Listing 3.6: Compiled language subdirectory outputs inside the build workspace.

This filesystem structure eliminates runtime string interpolation by serving raw HTML documents with pre-baked localized primitives directly to the client browser.

### 3.6.2. Infrastructure routing and request dispatching

Because the application compilation results in physically isolated subdirectories, traffic orchestration is managed directly at the infrastructure layer by the local Node.js reverse proxy (`prod-server.js`).

To handle this cross-origin traffic efficiently, the server is built upon the Express.js framework [7]. When an initial request targets the root domain, the proxy intercepts the connection and evaluates the client's HTTP `Accept-Language` request header. If the header explicitly presents English language tokens, the server internally maps the request path to the `/en/` subdirectory. For all other instances, or if the header metadata is absent, the proxy cascades to the Polish (`/pl/`) bundle as the system default, avoiding infinite circular redirection loops.

### 3.6.3. Client-server language synchronization

Modifying the language context via the user interface settings requires a full state synchronization across both the frontend client configuration and the backend persistence layer. Because the compilation bundles exist on isolated file paths, transitions cannot be resolved via internal Single Page Application client-side routing.

Instead, the operation is orchestrated via the `LanguageUtilsService`. Upon user confirmation of a language change, the service issues an asynchronous HTTP PATCH request to update the user's profile database state, synchronizes local browser storage flags, and forces a hard browser window reload using:

```
1 window.location.href = newLanguageUrlPath;
```

This execution lifecycle forces the browser to drop the active in-memory execution tree. The subsequent request to the modified URL prefix allows the `prod-server.js` proxy layer to instantly dispatch the target language bundle.

# Chapter 4

# Database design and data modeling

## 4.1. User management entity

The core of the user management module is the `Client` entity, which represents a registered customer within the system. To adhere to database normalization principles, the `Client` table stores only primary identifying attributes. It accommodates both individual and corporate customers by providing fields for personal names (`firstName`, `lastName`) as well as business entities (`organizationName`). Furthermore, it tracks security metrics such as failed login attempts (`failedLoginCount`), temporary account lockouts (`lockedUntil`), and the overall account `status`.

Contact information is structurally decoupled from the main client record. Phone numbers and email addresses are modeled as distinct entities (`ClientPhone` and `ClientEmail`) and are associated with the client through one-to-many relationships. This modular design natively supports scenarios where a single customer requires multiple authorized contact methods.

The system utilizes a stateless authentication architecture based on One-Time Passwords (OTPs). Instead of relying on a dedicated, highly volatile session table, the OTP verification parameters—specifically the generated `code` and its strict `expiresAt` timestamp—are persisted directly within the `ClientPhone` entity. This architectural decision minimizes database overhead during the authentication flow while implicitly validating the communication channel.

To fulfill legal compliance and manage user preferences, the data tier models many-to-many relationships using explicit associative entities. The `UserConsent` entity links clients to `ConsentType` definitions, tracking the boolean state of the agreement alongside a precise `grantedAt` timestamp for auditability. This table employs a composite primary key (`@EmbeddedId`) to enforce strict uniqueness at the database level. In a similar manner, associations for communication channels are managed by the `UserNotificationSetting` entity, which connects clients with configurable `NotificationType` records. However, unlike consents, notification settings utilize a surrogate auto-generated primary key to manage the associations.

Finally, the metadata definitions for both consents and notifications implement the system-wide localization pattern, utilizing `ConsentTypeTranslation` and `NotificationTypeTranslation` entities to serve dynamic descriptions in the user's preferred language.

## 4.2. Services and offers catalog

The system differentiates between active client subscriptions (services) and promotional marketing packages (offers). Both domains utilize a composite primary key translation pattern to support dynamic localization without altering the core database schemas.

The base capabilities of the operator are defined within the `Service`, `TVChannelPackage`, and `Hardware` entities. These tables act as standardized dictionaries. When a customer subscribes to an offering, the system generates a `ClientService` record. This associative entity bridges the `Client` and `Service` tables, encapsulating the entire lifecycle of the subscription by tracking its active status, start and end dates, and the next billing cycle. Because telecommunications services are inherently location-bound, this entity is strictly associated with a `ClientAddress` record, anchoring the digital service to a physical contract location.

Hardware provisions, such as routers or set-top boxes, are tracked through the `ClientHardware` entity. To maintain strict referential integrity, physical assets are linked directly to the specific `ClientService` rather than the client globally, alongside physical tracking metrics like serial numbers and ownership status.

For television services, the `TVChannel` entity stores channel pictograms directly as byte arrays (`ByteArray`) to ensure self-contained visual assets. Client subscriptions to specific channel bundles are recorded in the `ClientPackageSubscription` entity. To enforce data integrity and prevent duplicate active subscriptions, this table applies a composite `UniqueConstraint` on the `client_number` and `tv_channel_package_id` columns at the database level.

Parallel to the active services, the commercial catalog is partitioned into base offers and specific variants. The `Offer` entity serves as the aggregate root, defining the general marketing proposition, categorization, search tags, and legal metadata. Textual attributes such as titles and subtitles are isolated within the `OfferTranslation` table.

Specific pricing structures and technical configurations are delegated to the `OfferVariant` entity. A notable architectural decision within this entity is the use of a native text column (`blocksJson`) to store dynamic UI configuration blocks as JSON strings. Instead of creating numerous sparse relational tables to handle varied product parameters (e.g., internet transmission speeds, TV channel lists, or mobile data limits), the system leverages a schemaless structure. This hybrid approach provides maximum flexibility, allowing the frontend presentation layer to render complex, varied promotional components without requiring structural database migrations for every new marketing campaign. Finally, customer conversions are recorded in the `OfferOrder` associative entity, linking a client to a specific product variant alongside an exact transaction timestamp.

## 4.3. Billing history

The financial ledger of the system is modeled around three primary entities: `Invoice`, `Payment`, and `Refund`. This structure ensures a robust and immutable audit trail for all monetary transactions associated with a client.

The `Invoice` entity tracks the billing cycle, storing critical timestamps such as the issue date, due date, and the specific billing period boundaries. Its state lifecycle is governed by an enumerated `InvoiceStatus` (e.g., `UNPAID`, `PAID`, `PENDING`, `OVERDUE`). Furthermore, each invoice includes a `documentId` reference, which seamlessly links the relational database record to the generated PDF artifact stored within the system.

Inbound transactions are recorded in the `Payment` entity. This table captures the monetary amount, the gateway transaction identifier, and the specific payment method utilized by the

client, such as BLIK, payment card, or standard bank transfer. Notably, the foreign key linking a payment to a specific invoice is explicitly configured as optional (`nullable = true`). This architectural choice accommodates both direct invoice settlements and general account balance top-ups, providing flexibility in how client funds are allocated.

Outbound transactions and money returns are handled by the `Refund` entity. To maintain strict financial integrity, a refund cannot exist in isolation; it is rigidly bound to an original `Payment` record via a mandatory foreign key. The entity also records the execution date, the operational reason for the return, and its current status, ensuring that customer disputes and financial reversals are thoroughly documented.

## 4.4. Message and notification history

The system implements a dual-layered messaging architecture designed to efficiently handle both user-specific alerts and system-wide broadcasts.

Targeted notifications are modeled using the `PersonalMessage` entity, which establishes a direct many-to-one relationship with a specific client. To accommodate various notification contexts (such as maintenance alerts, system outages, or invoice issuances) without constantly altering the relational database schema, this entity utilizes a native PostgreSQL `JSONB` column (`payload`) handled via Hibernate's JDBC type code mapping. This hybrid document-relational approach permits the backend to persistently store arbitrary dynamic key-value pairs. These parameters are subsequently extracted and injected into localized text templates by the presentation layer at runtime, minimizing database storage overhead while ensuring high flexibility.

Conversely, system-wide broadcast announcements are managed using the `GlobalMessage` entity. To support multi-language distribution channels, textual attributes such as titles and contents are completely isolated within the `GlobalMessageTranslation` table, which utilizes an embedded composite primary key (`GlobalMessageTranslationId`) containing the target language code.

To ensure system scalability as the customer base grows, the architecture strictly avoids duplicating the core broadcast message content for every registered user. Instead, the message payload is written once, whereas individual user interactions are tracked through the `ClientGlobalMessageState` associative entity. This state table independently records granular user inbox actions, including read indicators (`isRead`), precise read timestamps (`readAt`), and soft-delete flags (`isDeleted`). To maintain strict transactional safety, the table enforces a database-level unique constraint (`uc_client_global_message_states`) on the combined client and global message foreign keys, preventing duplicate state initialization.

Finally, static system documents, such as Terms of Service and Privacy Policies, follow a unified localization blueprint. The `Document` entity serves as a stable string-based key identifier, while its multi-language titles and raw HTML text bodies are mapped through the `DocumentTranslation` entity. This guarantees that legal, regulatory, and technical documentation can be seamlessly retrieved according to the client's language preference.

# Chapter 5

# Server-side implementation (Backend)

## 5.1. Technology stack and framework selection

The application's backend was implemented as a Spring Boot service written in Kotlin. Build is automated using a multi-module Maven setup.

The version of Kotlin used is `Kotlin 2.3.10` and the project is compiled against a `Java 25` target. Kotlin was selected due to its concise syntax, built-in null-safety and interoperability with the Java ecosystem on which Spring is based. To reduce boilerplate, dedicated Kotlin compiler plugins `spring`, `jpa`, `all-open` and `no-arg` are enabled. Thanks to that Spring beans and JPA entities can be declared without manually marking classes as `open` or providing no-argument constructors.

The application uses `Spring Boot 4.0.3` and the following Spring modules:

- **Spring Web**: exposing the REST API through MVC controllers;
- **Spring Security**: request authorization and the authentication pipeline;
- **Spring Data JPA**: data access on top of the Hibernate ORM;
- **Spring Validation**: request validation based on Jakarta Bean Validation;
- **Spring Cache**: caching of selected read operations;
- **Spring Mail**: outgoing e-mail notifications.

Authentication is implemented with `Spring Security` using a one-time password (OTP) login flow. If verification is successful, a stateless JSON Web Token (JWT) is issued, generated and validated with the `JJWT 0.12.6` library. Subsequent requests are authorized using the bearer token, which removes the need for server-side session state.

## 5.2. Architectural patterns and core modules

Each module encapsulates a single architectural concern:

- `domain`: the core of the system, containing the persistent entities (e.g., `Client`, `Service`, `Invoice`, `Offer`) expressed as JPA-mapped classes. It holds no business logic dependent on outer layers.

- `infrastructure`: the persistence layer, providing Spring Data repositories and the database migration configuration.
- `service`: the business-logic layer, containing application services, the data-transfer objects (DTOs) exposed to the outside world, and the mappers that translate between entities and DTOs.
- `api`: the presentation layer, exposing the REST controllers, request validation, locale resolution and centralized exception handling.
- `application`: the composition root, which assembles the modules, provides cross-cutting configuration (security, OpenAPI) and contains the Spring Boot entry point that produces the executable artifact.

## 5.3. API design and endpoints

- **Authentication**
  - `POST /api/auth/login`: Send a login OTP via SMS; always returns 200 to prevent enumeration.
  - `POST /api/auth/register`: Send a registration OTP after matching client number, phone and email.
  - `POST /api/auth/verify-code`: Verify the OTP and return a JWT; activates new accounts.
  - `GET /api/auth/dev/code`: Return the current OTP for a phone number (development only).
  - `GET /api/auth/validate-token`: Check whether the supplied token is valid.
- **Auth Documents**
  - `GET /api/auth/terms-of-service`: Terms of service document for the registration flow.
  - `GET /api/auth/privacy-policy`: Privacy policy document for the registration flow.
  - `GET /api/auth/marketing-consents`: Marketing consents document for the registration flow.
- **Clients**
  - `GET /api/clients`: Return the authenticated client's profile.
  - `GET /api/clients/phone-numbers`: Return all phone numbers of the authenticated client.
  - `PUT /api/clients`: Update the authenticated client's profile fields.
- **Consents**
  - `GET /api/clients/consents`: Return all consent types with the authenticated client's current status.
  - `PUT /api/clients/consents`: Bulk update the authenticated client's consent choices.

- **Notification Settings**
  - `GET /api/clients/lang-notification-settings`: Return notification types with the authenticated client's preferences.
  - `PUT /api/clients/lang-notification-settings`: Bulk-update the authenticated client's notification preferences.
- **Billing**
  - `GET /api/billing`: Current billing balance and account number for the authenticated client.
  - `GET /api/billing/invoices`: All invoices for the authenticated client.
  - `GET /api/billing/invoices/{invoice_id}`: Detailed breakdown of a single invoice.
  - `POST /api/billing/payments/{invoice_id}`: Process a payment for a specific invoice.
  - `GET /api/billing/payments`: History of successful and pending payments.
  - `GET /api/billing/returns`: List of processed, accepted and rejected returns.
  - `POST /api/billing/returns`: Create a money-return request.
  - `GET /api/billing/transfer-details`: Bank transfer receiver, address and account number.
- **Services**
  - `GET /api/services`: All services assigned to the authenticated client.
  - `GET /api/services/{service_id}`: Detailed service info incl. hardware and TV packages.
  - `GET /api/services/{service_id}/packages`: All TV channel packages for a TV service.
  - `GET /api/services/{service_id}/packages/{package_id}`: A single TV channel package.
  - `GET /api/services/addresses`: All addresses associated with the client.
  - `POST /api/services/{service_id}/packages/{package_id}`/init: Start two-step package subscription (sends OTP).
  - `POST /api/services/{service_id}/packages/{package_id}/verify`: Verify OTP and subscribe to the package.
  - `DELETE /api/services/{service_id}/packages/{package_id}`: Unsubscribe from a TV channel package.
- **Services Documents**
  - `GET /api/services/promotions-regulations`: Promotion regulations document for package purchase.
- **Hardware**
  - `GET /api/hardware`: All hardware items across the authenticated client's services.

  - `GET /api/hardware/{hardware_id}`: A single hardware item owned by the authenticated client.

- **Offers**
  - `GET /api/offers`: All offers on the offers main page.
  - `GET /api/offers/{offer_id}`: Offer details including variants and blocks.
  - `POST /api/offers/{offer_id}/variants/{variant_id}/init`: Start two-step order verification (sends OTP).
  - `POST /api/offers/{offer_id}/variants/{variant_id}/verify`: Verify OTP and place the order.

- **Messages**
  - `GET /api/messages/global`: Global messages with per-client read state.
  - `PUT /api/messages/global/{message_id}`: Mark a global message as read.
  - `DELETE /api/messages/global/{message_id}`: Delete a global message.
  - `GET /api/messages/personal`: Personal messages addressed to the authenticated client.
  - `PUT /api/messages/personal/{message_id}`: Mark a personal message as read.
  - `DELETE /api/messages/personal/{message_id}`: Delete a personal message.

- **Settings**
  - `GET /api/settings/consents`: All marketing and legal consents, localized.
  - `PUT /api/settings/consents`: Update the status of a specific consent.

- **Settings Documents**
  - `GET /api/settings/privacy-policy`: Privacy documents and consents grouped by category.
  - `GET /api/settings/terms-of-service`: Hierarchical terms-of-service sections.

- **Support**
  - `GET /api/contact-help`: Contact phone and email with schedules.
  - `GET /api/faq/categories`: List of all FAQ categories.
  - `GET /api/faq/categories/{route}`: FAQ page data for a specific category route.
  - `GET /api/faq/search`: Search FAQ questions by a query string.

- **Dashboard & Layout**
  - `GET /api/dashboard`: Aggregated dashboard data (summary, messages, services, offers).
  - `GET /api/layout`: Header data such as client name and message count.

- **Documents**
  - `GET /api/documents`: List of all documents (id and name), localized.
  - `GET /api/documents/{doc_id}`: A single document with full content.

  - `GET /api/documents/{id}/text`: Raw HTML body of a document.
  - `GET /api/documents/{id}/pdf`: Download the document as a PDF file.

- **Public Documents**
  - `GET /api/privacy-policy`: Public grouped privacy policy documents.
  - `GET /api/terms-of-service`: Public grouped terms of service documents.

## 5.4. Data access layer

Data persistence in the application is implemented using `Spring Data JPA` with `Hibernate` as the underlying ORM provider and a `PostgreSQL 17` database. Data is accessed through repository interfaces extending `CrudRepository` or `JpaRepository`. Query logic is derived automatically by Spring from method names. This removes the need for hand-written SQL for the majority of operations. Queries are consistently scoped by the client identifier, so that data isolation between clients is enforced at the data-access level rather than relying on the controller layer.

Automatic schema generation by Hibernate is disabled, as instead, the database schema is defined explicitly through `Liquibase` changelogs. The master changelog gathers a number of modular changelog files and a sequence of incremental change sets that create tables, add foreign key and uniqueness constraints, and perform later refactorings such as renaming tables or relaxing nullability. This approach keeps the schema under version control and makes migrations reproducible across environments. Liquibase contexts are used to separate concerns within the changelog.

Invoice and document PDFs are generated server-side from the relational records using the `OpenPDF` library.

## 5.5. Integration with presentation layer

The backend exposes a stateless REST API consumed by a single-page web application. Communication uses JSON over HTTP, with serialization and deserialization handled by the `Jackson` library; the public contract is expressed only through DTOs rather than persistent entities, which decouples the external representation from the internal data model. Cross-origin requests from the frontend application are permitted through an explicit CORS configuration that whitelists the client origin, the allowed HTTP methods and credentialed requests.

## 5.6. Interactive API documentation

To keep the API documentation accurate and continuously synchronised with the implementation, the project uses `springdoc-openapi`, which generates an `OpenAPI 3` specification directly from the source code at runtime and serves an interactive Swagger UI. The documentation endpoints (`/v3/api-docs` and `/swagger-ui`) are explicitly exempted from authentication so that the interface remains browsable during development. The controllers are annotated with OpenAPI metadata: `@Tag` to group related endpoints, `@Operation` to provide human-readable summaries and descriptions, and `@ApiResponse` together with `@Schema` to document the possible response codes and their payload structures.

# Chapter 6

# System testing and quality assurance

## 6.1. Unit testing

he lowest layer of the backend testing strategy consists of unit tests that exercise individual components in isolation from the Spring container, the database, and the network. In the context of this project a *unit* is a single service-layer class—such as the billing, authentication, messaging, or document service—together with the mappers that translate between persistent entities and the data-transfer objects exposed by the API. Because these classes concentrate the business logic of the application, including balance computation, validation rules, status transitions, and DTO assembly, they are the components for which fast and deterministic tests provide the highest value. Their collaborators, namely the Spring Data JPA repositories and auxiliary services such as the SMS gateway, are external to the unit and are therefore replaced by test doubles, so that each test asserts the behaviour of exactly one class.

The unit tests are written in Kotlin and executed on the JUnit 5 platform through the `kotest-runner-junit5` adapter, which integrates them with the standard Maven Surefire build phase. Two libraries form the core of the toolchain:

- **Kotest** provides the test framework and the assertion library. Tests are expressed in the behaviour-driven `DescribeSpec` style, in which a `describe` block names the method under test and a nested `it` block describes a single observable behaviour. Assertions use the fluent infix matchers of `kotest-assertions-core` (for example `shouldBe`, `shouldContain`, `shouldHaveSize`, and `shouldThrow`), which keep the expected outcome readable as a natural-language statement.
- **MockK**, a mocking library designed for Kotlin, is used to create the test doubles that stand in for the repositories and collaborating services. Stubbed responses are declared with the `every { ...} returns ...` construct, interactions are confirmed with `verify`, and argument values passed to collaborators are inspected through captured slots.

The unit tests reside in the `service` Maven module, mirroring the package layout of the production code. The suite comprises ten specifications, one per service or mapper under test, and a total of more than one hundred individual test cases. Each specification follows a consistent structure that keeps the cases independent and their intent explicit. The mocks and the system under test are instantiated once at the top of the specification, and a `beforeEach` hook invokes `clearAllMocks()` before every case, so that no stubbing or recorded interaction leaks from one test into the next and the cases can be run in any order. Recurring domain

objects, such as clients, invoices, and payments, are produced by small local builder functions with default arguments; a test overrides only the fields relevant to the scenario it describes, for example a paid invoice or a client with no surname, which removes boilerplate and makes the meaningful variation in each case immediately visible. Cases are grouped by the public method they target, and each case name states the expected behaviour in full, so that the names alone document the contract of the service layer.

Across the suite the tests apply a small, repeated set of techniques that together cover both the values a service returns and the side effects it produces. State-based assertions check the result of a call, such as the computed account balance, the contents of a returned list, or a fallback display name when optional client fields are absent. Interaction-based verification confirms that a service collaborates with its repositories as expected—for example, that a successful payment is persisted exactly once and the corresponding invoice is saved with its status changed. Argument capture records the object handed to a repository so that the test can assert on the data the service constructed, rather than only on the value it returns. Exception testing asserts that invalid input and missing entities raise the correct exception type: `NoSuchElementException` for absent records, `IllegalStateException` for forbidden transitions such as paying an already-settled invoice, and `IllegalArgumentException` for invalid amounts. Finally, relaxed mocks are used for collaborators whose return value is irrelevant to the behaviour under test, such as the SMS gateway during authentication, so that only the interactions that matter need to be stubbed explicitly.

Listing 6.1 illustrates the resulting style on a representative case from the billing service. The repositories are stubbed to return a fixed set of domain objects, the service method is invoked, and the assertions express the expected outcome directly: the computed balance and a substring of the formatted account number.

```
describe("getBillingSummary") {
    it("computes negative balance as standalonePayments - unpaidTotal") {
        every { clientRepository.findByClientNumber("C001") } returns client()
        every { invoiceRepository.findAllByClientClientNumber("C001") } returns listOf(
            invoice(1, total = BigDecimal("100.00")),
            invoice(2, total = BigDecimal("50.00")),
            invoice(3, total = BigDecimal("999.00"), status = InvoiceStatus.PAID),
        )
        every { paymentRepository.findAllByClientClientNumber("C001") } returns listOf(
            payment(amount = BigDecimal("20.00"), invoice = null),
            payment(amount = BigDecimal("999.00"), invoice = invoice(99)),
        )

        val summary = service.getBillingSummary("C001")

        summary.balance shouldBe BigDecimal("-130.00") // 20 - 150
        summary.accountNumber shouldContain "00000001"
    }
}
```

Listing 6.1: A unit test of the billing service, showing repository stubbing with MockK and fluent Kotest assertions.

By isolating the service layer behind mocked collaborators, these tests run in milliseconds and without any external infrastructure, which makes them suitable for continuous execution during development and as part of the automated build. They pin down the application's

business rules at a fine granularity and act as executable documentation of the expected behaviour, while the correctness of the wiring between layers and the persistence configuration is covered by the integration tests described in the following section.

## 6.2. Integration testing

Whereas the unit tests isolate a single service class behind mocked collaborators, the integration tests exercise a complete vertical slice of the backend with its components wired together by the Spring container. A single test drives the application through its public HTTP boundary and follows the request down through the security filter chain, the controller, request validation, the service layer, the Spring Data repositories, and finally a real database, before the response is serialized back to JSON. Their purpose is therefore complementary to that of the unit tests: rather than the business rules of an individual class, they verify the correctness of the *wiring* between the layers and of the parts that a mock cannot represent—the JPA mappings, the Liquibase schema, the derived queries, the Jackson serialization contract, locale resolution, and the security configuration.

The tests are written in Kotlin in the same `DescribeSpec` style and with the same Kotest assertion library as the unit tests. A `@SpringBootTest` context boots the full application, and `@AutoConfigureMockMvc` provides a `MockMvc` instance that dispatches requests through the entire server-side MVC stack—routing, validation, the security filters, and the centralized exception handler—without binding a network socket. The database is a real `PostgreSQL 17` instance provisioned through Testcontainers and started once for the whole suite, so that the migrations, entity mappings, and PostgreSQL-specific features such as `JSONB` columns are exercised exactly as deployed. The Kotest Spring extension injects collaborators such as `MockMvc`, the `ObjectMapper`, the repositories, and the `JwtTokenProvider` into each specification.

The environment is kept hermetic and deterministic. Liquibase runs under a `test` context that builds the schema but omits the development seed, so each test starts from an empty database and inserts only the rows it needs; e-mail and SMS delivery are disabled so that no external service is contacted; and a dedicated `security-test` profile reproduces the production filter chain but signs tokens with a fixed key, letting each test mint valid, expired, or malformed tokens deterministically. The seventeen specifications reside in the `api` module and contain over one hundred and eighty cases, complemented by a security-configuration specification in the `application` module. Each extends a common `BaseIntegrationTest` that centralizes this setup. Because the database is shared and stateful, every specification resets the relevant tables in a `beforeTest` hook—deleting in an order that respects the foreign-key constraints—which keeps the cases independent and order-agnostic, while small builder functions such as `savedClientWithToken` and `savedInvoice` seed each scenario and return the managed entities together with a fresh token.

Each case asserts on the HTTP contract—the status code and, through `jsonPath` expressions, the structure and `snake_case` field naming of the JSON body—and, for requests that mutate state, additionally reads the affected rows back through the repository to confirm the side effect was durably applied. Listing 6.2 illustrates the contract side on a locale-sensitive read: the test seeds a Polish-language client together with a translated document and confirms that an explicit `Accept-Language` header overrides the client's stored language, so that the English translation is served—validating the locale-resolution logic and the translation mappings end to end.

```
it("uses Accept-Language header when provided, ignoring client's stored
    language") {
    self.clientRepository.save(Client(clientNumber = "C001", languageCode = "
    pl"))
    self.documentRepository.save(testDocumentWithTranslations())
    val token = self.jwtTokenProvider.generateToken("C001")

    self.mockMvc.get("/api/documents/Doc1") {
        accept = MediaType.APPLICATION_JSON
        header("Authorization", "Bearer $token")
        header("Accept-Language", "en")
    }.andExpect {
        status { isOk() }
        jsonPath("$.name") { value("Document 1") }
        jsonPath("$.text") { value("English content") }
    }
}
```

Listing 6.2: An integration test of locale resolution, confirming that the `Accept-Language` header overrides the client's stored language.

Beyond the per-endpoint happy paths, the suite targets behaviour that only emerges once the layers are assembled: a dedicated specification checks every protected endpoint against missing, malformed, and expired tokens; the OTP login and registration flows are exercised end to end, including enumeration protection and account lockout after repeated failures; the two-step `init`/`verify` operations confirm that an invalid code returns `422` rather than logging the user out; ownership checks confirm that one client cannot access another's invoice; mutating endpoints such as payment are verified by reading the persisted records back; and a specification in the `application` module confirms the CORS, stateless-session, and public-documentation behaviour of the security configuration.

These tests are necessarily slower and fewer than the unit tests, since each boots the Spring context and exercises a real database, but they validate precisely what the unit tests cannot: that the controllers, services, repositories, schema, and security filters behave as a coherent whole. Like the unit tests, the suite runs automatically in the continuous-integration workflow on every push.

## 6.3. Functional testing

The functional testing phase verifies that the integrated system complies with the defined business workflows and user journey specifications. To make the tests reproducible, the procedures are structured as a deterministic User Acceptance Testing (UAT) suite. This allows external evaluators to verify the operational state of the deployment environment by following sequential execution paths.

### 6.3.1. Environment and prerequisites

To replicate the test results locally, the evaluator must first follow the installation instructions specified in Appendix A. Prior to executing the functional test suite, the evaluator must ensure that the following preconditions are met:

- The local frontend production build (`prod-server.js`) is initialized and running on port `3000`.
- The browser cache is cleared to prevent token interference from previous sessions.

- The local backend server is installed and running on port `8080`.
- The test database contains the standard seeding profile, providing the mock credentials used in the test scenarios below.

### 6.3.2. Functional test matrix

The application includes over 50 individual screens with separate functions. To concisely document all the critical paths an end user may take when using the app, the verification layout maps the primary user journeys into a structured validation matrix. Table 6.1 contains step-by-step instructions, expected outputs, and pass criteria for testing the application locally using examples of critical functional paths in every section of the app.

Table 6.1: System functional verification and User Acceptance Testing matrix.

| Functional Target | Step-by-Step Instructions | Expected Verification Result | Status |
|---|---|---|---|
| **Login** | 1. Navigate to `/start`.<br>2. Click "Login".<br>3. Submit test phone number in the form.<br>4. In the terminal where the backend server was initialized, find the generated code in the last entry, enter the code in the app.<br>5. Submit form. | Browser updates URL to `/dashboard` and displays user metrics. | Pass |
| **Invalid OTP code handling** | 1. Navigate to `/start`.<br>2. Click "Login".<br>3. Submit test phone number.<br>4. Submit an invalid code. | An HTTP request with response status 422 can be visible in the Network tab of DevTools. | Pass |
| **Route guard enforcement** | 1. In DevTools, Clear session storage.<br>2. Manually enter `/dashboard` into browser address bar. | Navigation is intercepted; browser redirects to `/start`. | Pass |
| **URL language swap** | 1. Open the app.<br>2. Change "/en/" in the URL to "/pl/" or vice versa. | Window reloads; the content language changes to the language in the link. | Pass |
| **Unchanging database language setting** | 1. Log into the app, navigate to `/settings/lang-notif`.<br>2. Change "/en/" in the URL to "/pl/" or vice versa. | The default language in the settings doesn't change. | Pass |
| **Persistent settings** | 1. Log into the app, navigate to `/settings/lang-notif`.<br>2. Make changes to the notification choices and/or language.<br>3. Click "Save changes". | Window reloads, the settings have been saved to the database. | Pass |

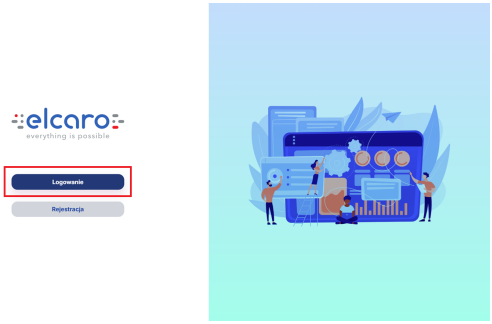
*Continued on next page*

Table 6.1 – Continued from previous page

| Functional Target | Step-by-Step Instructions | Expected Verification Result | Status |
|---|---|---|---|
| **Dashboard shortcuts** | 1. Log in as the test user.<br>2. In the "Payments" section of the dashboard, click "Pay". | The router navigates to `/billing/unpaid`, where invoices can be selected with checkboxes. | Pass |
| **Submitting a return request** | 1. Log in with the test phone number +48600700800, test code 7777.<br>2. Navigate to `/billing`.<br>3. Click the "Return" button.<br>4. Submit a "return request" for 5 PLN. | The request is submitted, which is indicated by the green pop-up in the top right of the screen. | Pass |
| **Modifying a TV package** | 1. Log in with the test phone number +48123456789.<br>2. Navigate to `/services`.<br>3. Click on the "Comfort Plus 4K" (pl: "Komfort Plus 4K") service.<br>4. Click "Add package" on "Filmy+".<br>5. Choose the phone number +48888888888 and click the checkbox.<br>6. In the terminal where the backend server was initialized, find the generated code in the last entry, and submit the code in the app. | A "Package has been successfully activated" message is visible. | Pass |
| **Making an order** | 1. Log into the app.<br>2. Scroll down in the dashboard and click on any of the offers.<br>3. Click "Order" on one of the offer variants.<br>4. Choose the phone number +48123456789.<br>5. Enter the test code 5555. | After a few seconds, a success message will pop up. Note: The email can only be sent from the server where the app is deployed. | Pass |
| **Using a search filter in FAQ** | 1. Log into the app.<br>2. Navigate to `/help/faq`.<br>3. Type "router" in the search bar and hit Enter. | The "No internet" question pops up, with the keyword "router" in the text below. | Pass |
| **Checking messages as read** | 1. Log into the app.<br>2. Navigate to "/messages/global".<br>3. Click on "Privacy policy update". | The opened envelope symbol is visible in the top right of the message. | Pass |

### 6.3.3. End-to-End user journey case study: transfer details path

Figure 6.1 showcases a test scenario where the test user logs in, navigates to the billing section of the app, selects an invoice to pay, and views the suggested details to enter when creating a bank transfer.



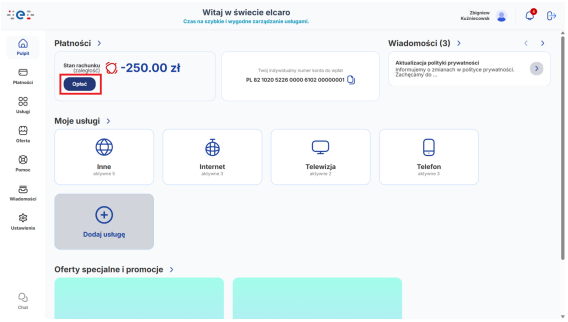

(a) Step 1: Public landing interface.



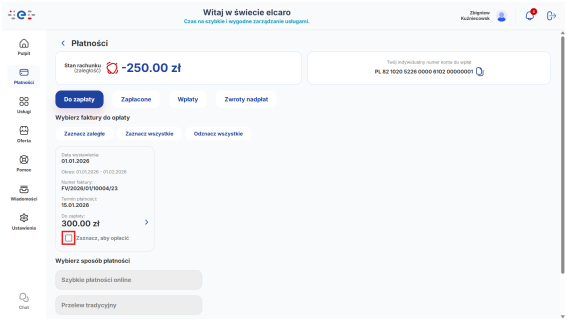

(b) Step 2: Enter the test phone number.



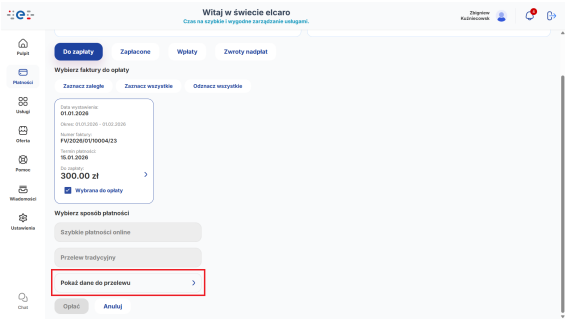

(c) Step 3: Enter the test code.



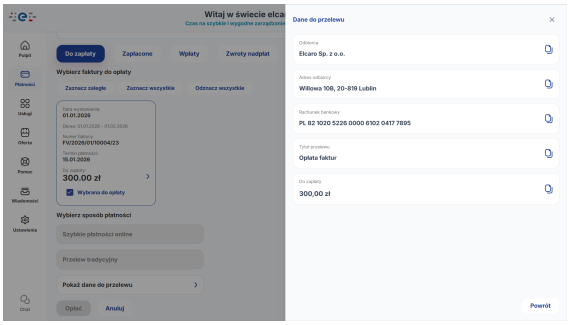

(d) Step 4: Click the dashboard pay button.




(e) Step 5: Select the invoice to pay.




(f) Step 6: Click the transfer details button.




(g) Step 7: Transfer details are shown.

Figure 6.1: Sequential 7-screen visual audit log documenting the path to show suggested transfer details to pay selected invoices.

# Chapter 7

# Project management and task allocation

## 7.1. Division of work

**Pavel Charheika**

During the initial phases of the project, I actively contributed to the system architecture and database design. My involvement included supporting the design of the Entity-Relationship Diagram (ERD), and I was specifically responsible for assembling all components and generating the final, complete ERD for the application. On the backend, I authored the corresponding domain entities and expanded the database structure via domain-specific Liquibase schema migrations, managing complex data seeding profiles, systematic generation of comprehensive localized mock data, and strict migration dependencies.

Within the backend code for the main application, I developed a substantial part of the business logic and server-side infrastructure, focusing on several key modules:

- **Payments and Billing:** Engineered the RESTful endpoints and server-side logic for the complete payment domain, defining the core domain entities. Developed the underlying business logic and API contracts for handling billing operations and transaction processing.
- **Account Settings and Preferences:** Developed the APIs and business logic for managing legal consents and account configurations. Implemented a symmetric data mapping architecture for contact methods and a notification matrix to manage user communication preferences.
- **Dashboard and Global Layout:** Developed aggregated endpoints to synchronize core user metrics and dynamic state data for the frontend layout. Additionally, integrated a two-step verification mechanism for secure service activations.
- **Document Processing:** Designed the server-side PDF engine using HTML templates for automated invoice and document generation. Secured download access paths and enabled internationalization for localized rendering.
- **Localization and Messaging:** Implemented a database-driven translation architecture for system messages, service offers, and UI consent descriptions. This included standardizing backend locale resolution from HTTP requests and engineering a backend-driven user interface layer using PostgreSQL `JSONB` payloads.

- **Support and Customer Service:** Developed the Support and FAQ modules, designing and securing the public endpoints with language-based filtering to ensure proper localization of user assistance.

- **Quality Assurance and API Documentation:** Authored integration test suites for the business modules developed, ensuring reliability of system flows. Furthermore, structured the OpenAPI (Swagger) documentation to specify API responses, security requirements, and data contracts specifically for the implemented endpoints.

In addition to the software implementation, I actively participated in the preparation of this document. I authored Chapter 4, as well as sections 2.3 and 2.4.

**Franciszek Skawiński**

At the outset of the project I collaborated with the other team members on the design of the system's Entity-Relationship Diagram (ERD). Working from the specifications received from the client, I selected Kotlin as the backend programming language and established the project scaffolding, defining the multi-module structure that enforces a strict, inward-pointing dependency chain between the `domain`, `infrastructure`, `service`, `api` and `application` layers.

Within the backend code for the main application I authored a substantial part of the server-side infrastructure and business logic, focusing on several key modules:

- **Security and Authentication:** Implemented the security and authentication layer using Spring Security, configuring stateless JWT-based authentication, the login flow and endpoint protection. Built the user-registration module with verification of contact details and integrated a One-Time Password (OTP) mechanism to secure sensitive operations.

- **Customer Identity and Preferences:** Modelled the customer-identity domain (clients together with their associated phone numbers and email addresses) and developed the business logic for managing legal consents and notification settings.

- **Service Catalogue and Hardware:** Engineered the domain-services layer, developing the endpoints and logic for the service catalogue, client subscriptions, TV channel packages and the associated hardware.

- **Messaging and Offers:** Developed the messaging module, covering global and personal messages with per-client read state, and implemented the offers module exposing promotional packages and their variants.

- **Communications and Documents:** Implemented outbound communications, including the application-side configuration for transactional email and SMS delivery, and developed the document endpoints together with secured, locale-aware document download access.

- **Persistence Layer:** Authored the persistence-agnostic domain entities, grouped by bounded context, together with the Spring Data repositories that expose them.

- **Cross-cutting Concerns:** Implemented the global exception handler that maps domain and application exceptions to consistent HTTP responses across all endpoints.

- **Quality Assurance:** Authored extensive unit and integration test suites covering the services, mappers and REST endpoints I implemented, ensuring the reliability of the system flows.

Besides the main application code I carried out the following supporting work:

- prepared the files required to launch the application and its database as Docker containers, allowing the application to be started quickly and tested under conditions similar to the deployment environment;
- created the GitHub Actions workflow that automatically verifies that all tests pass after every commit pushed to the repository;
- configured the Amazon Web Services Simple Email Service to enable sending emails from the application;
- configured the Twilio messaging service to enable sending SMS messages from the application;
- prepared the Liquibase change logs used for database migrations and the development seed data.

In addition to the implementation work, I created the scaffolding of the LaTeX file for the thesis, drafted its first outline and was responsible for the final formatting. Within the thesis I authored Chapter 5 as well as sections 6.1 and 6.2.

**Yevhenii Tarasiuk**

- **Architectural design and organization**
  - **Database schema engineering:** Collaborated on the design of the system's Entity-Relationship Diagram (ERD), specifically defining the relational schemas, constraints, and data attributes for the core Services and Offers modules.
  - **API contract specification:** Established the OpenAPI specification pipeline to formalize client-server communication. Created a significant part of the endpoint contracts to guide backend development. Configured the Prism tool to create mock API endpoints with example responses using the contracts.
- **Frontend infrastructure and middleware engineering**
  - **Reverse proxy architecture:** Developed a custom production reverse proxy framework (`prod-server.js`) to manage cross-origin traffic.
  - **Security and access control:** Engineered the client-side authorization subsystem and the global `authGuard` routing logic to enforce security boundaries over protected application states.
  - **Global exception handling:** Implemented an HTTP error interceptor middleware layer to centrally capture failed asynchronous requests and manage fallback routines, connecting it to UI notifications.
  - **Internationalization pipeline:** Aided in configuring the localized application routing logic and implemented the `LanguageUtilsService` to synchronize state and execute hard context shifts between compile-time language bundles. Participated in the textual translation and localization mapping of the user interface.

- **Feature modules implementation**
  - Co-authored the complete client-side presentation layer, primarily implementing the three critical business modules:
    * **Billing section:** Engineered the module architecture, developed data structures to display historical invoices, configured view filters to sort billing ledger statuses (such as paid and pending items), and implemented the asynchronous byte-stream download routine for generated PDF receipts.
    * **Services section:** Engineered the control panel and a One-Time Password verification system, allowing users to view individual service details and make authenticated requests to modify their services..
    * **Offers section:** Developed the primary customer-facing presentation layer, utilizing a responsive layout to display promotional packages, designing contextual filtering hooks to narrow down option selections dynamically based on user preferences. Implemented a highly customizable and modular offer variant menu.
- **Testing and documentation**
  - **Thesis contribution:** Authored Chapter 3 of this document (Client-side implementation) and handled the functional tests.

**Bogdan Vysocki**

I actively contributed to the project from its initial development phase. My involvement included supporting the design of the Entity-Relationship Diagram (ERD), with a focus on the Issues and Messages modules.

On the frontend side, I implemented the main application layout, including the header and sidebar, and ensured their responsiveness across different screen sizes. I also developed a set of reusable global UI components, including buttons, textfields, dropdowns, tabs, and other interface elements. In addition, I defined and applied the global color system throughout the project.

Furthermore, I installed and configured the `i18n` internationalization library and contributed to the translation process. I implemented shared `document-service` module, enabling consistent document reading and downloading functionality across the application.

I developed a substantial part of the client-side presentation layer, focusing on several key business modules:

- **Dashboard section:** Implemented an interface that aggregates selected sections from other core components. I also introduced queryParams support to enable direct navigation to specific application states, such as messages or services filters.
- **Help section:** Implemented browser-native `mailto` and `tel` integrations, enabling users to initiate email communication or phone calls directly from the application. I developed a Frequently Asked Questions (FAQ) subsection organized into thematic categories and designed a question search mechanism to improve user accessibility and navigation.
- **Messages section:** Developed functionality for viewing messages and notifications, implemented message deletion confirmation mechanisms, and ensured the module's responsiveness.

- **Settings section:** Implemented functionality allowing users to modify their primary personal data, as well as add and remove phone numbers and email addresses. I also developed language and notification settings, document subsections, hardware-related subsection with category-based filtering and detailed view, and the management of marketing consents.

In addition, I designed some of the endpoint contracts to support backend implementation. I also actively participated in the preparation of this document.

# Chapter 8

# Conclusion

## 8.1. Summary of project outcomes

The goal of this thesis was to design and implement a self-service web portal for customers of a telecommunications operator. The system was intended to provide users with a single digital access point for routine operations related to services, billing, payments, notifications, help materials, and account settings.

As part of the project, a client-server web application was developed. The frontend application was implemented using Angular and provides the user interface, routing, access control, communication with the backend, and localization support. The backend application was implemented as a Spring Boot service written in Kotlin and exposes a REST API responsible for processing requests, applying business logic, validating data, and accessing persistent storage. The database model includes the main entities required by the portal, such as users, services, offers, billing history, messages, and notifications. The project also included the preparation of the deployment environment and basic testing of the implemented functionality.

The resulting system demonstrates how routine customer service operations can be moved to a self-service model. This will reduce the number of repetitive support requests, improve access to customer information, and support more efficient communication between the operator and its users. Overall, the implemented portal provides a functional foundation for further development of a customer self-service platform. It separates frontend, backend, and data-related responsibilities, which makes the system easier to maintain, test, and extend in the future.

## 8.2. Future enhancements

At the beginning of the project, the industry partner provided initial Figma prototypes for a broader version of the customer portal. These materials included additional support-related functionality, especially an issue management module and an online support chat. An initial entity-relationship diagram (ERD) was also prepared for the issues management module. However, due to time constraints, these features were excluded from the final implementation and left as future work.

One possible extension is the implementation of the issues management module. This feature would allow customers to create and track support issues directly through the portal. Such issues could include complaints, failure reports, service enquiries, or other requests requiring operator assistance. Each issue could contain information such as its identifier, category, status, submission date, related service, and message history. Since an initial ERD for

this module already exists, it could serve as a foundation for extending the current database model.

Another planned enhancement is the online support chat. This feature would allow customers to contact a consultant directly from the portal when the available self-service options or FAQ materials are not sufficient. In later versions, the chat could also be extended with automatic responses for common questions or escalation mechanisms for more complex issues.

Although these features were outside the scope of the current implementation, the provided prototypes and data model make them natural candidates for future development. Their implementation would extend the portal from a self-service application into a more complete customer support platform.

# Appendix A

# Installation instructions

This appendix provides a step-by-step guide to installing, executing, and disabling the application ecosystem on a local evaluation host.

## A.1. System prerequisites

Before initialization, ensure that the host machine has the following software runtimes installed globally:

- Node.js (v24.13.0 or higher)
- npm (v11.9.0 or higher)
- Docker Desktop (v29.2.0 or higher, with Docker Compose enabled)

## A.2. Frontend application initialization

To deploy and serve the client-side presentation layer locally, execute the following instructions:

1. Clone the latest revision of the frontend repository from the official repository host:
   `https://github.com/elcaro-pl/elcaro-bok-app`
2. Install the Angular CLI engine (v21.1 or higher) globally on the host system to allow the `ng` command framework:
   ```
   npm install -g @angular/cli
   ```
3. Open a terminal instance in the root directory of the cloned workspace and install the required node module dependencies:
   ```
   npm install
   ```
4. Execute the production compilation sequence alongside the local reverse proxy server using the unified execution chain:
   ```
   ng build --localize && node prod-server.js
   ```
5. Once compilation resolves, launch a browser instance and navigate to:
   `http://localhost:3000`.

## A.3. Backend and database initialization

To locally initialize the backend server and the supporting database infrastructure, follow these steps:

1. Clone the latest revision of the backend repository from the official repository host:

   `https://github.com/elcaro-pl/elcaro-bok-app-be`

2. Launch a terminal instance with elevated execution permissions in the root directory of the backend workspace.

3. Trigger the automated orchestration script to build the required Docker containers and boot the application context. For Windows:

   ```
   ./start.ps1
   ```

   For Linux:

   ```
   ./start.sh
   ```

## A.4. Database state reset

If the local database schema or seeded records become corrupted during evaluation testing, the persistent storage volumes can be wiped and returned to their factory baseline:

1. Terminate the active backend application process (see Section A.5).

2. Within the backend root directory terminal, execute the container tear-down command alongside the volume destruction flag:

   ```
   docker compose down -v
   ```

3. Re-run the startup script (`./start.ps1`) to re-initialize a new instance of the database.

## A.5. Environment termination

To safely stop the application runtime environments and release bound network ports on the host machine, follow these termination procedures:

### Frontend termination

Select the active terminal instance hosting the Node.js proxy server and issue a standard interrupt signal via the keyboard shortcut: `Ctrl + C`.

### Backend termination

Select the active terminal running the backend runtime context and issue the identical interrupt signal (`Ctrl + C`). To clean up lingering containers globally, navigate to the backend root directory and execute:

```
docker compose down
```

# Bibliography


[1] Google. *Angular v21 Documentation: Routing*. Retrieved June 2026, from `https://v21.angular.dev/guide/routing`.

[2] Google. *Angular v21 Documentation: Angular Signals*. Retrieved June 2026, from `https://v21.angular.dev/guide/signals`.

[3] Google. *Angular v21 Documentation: HTTP*. Retrieved June 2026, from `https://v21.angular.dev/guide/http`.

[4] Google. *Angular v21 Documentation: Internationalization (i18n)*. Retrieved June 2026, from `https://v21.angular.dev/guide/i18n`.

[5] Stoplight. *Prism documentation: HTTP mocking*. Retrieved June 2026, from `https://docs.stoplight.io/docs/prism/83dbbd75532cf-http-mocking`

[6] PrimeTek. *PrimeNG documentation*. Retrieved June 2026, from `https://primeng.org`

[7] OpenJS Foundation. *Express.js Documentation: Routing and Middleware*. Retrieved June 2026, from `https://expressjs.com/en/guide/routing/`.

[8] Axios Contributors. *Axios Request Config: responseType*. Retrieved June 2026, from `https://axios.rest/pages/advanced/request-config#responsetype`.

[9] VMware Tanzu. *Spring Boot Reference Documentation*. Retrieved June 2026, from `https://docs.spring.io/spring-boot/index.html`.

[10] VMware Tanzu. *Spring Framework Documentation: Transaction Management and Cache Abstraction*. Retrieved June 2026, from `https://docs.spring.io/spring-framework/reference/`.

[11] VMware Tanzu. *Spring Data JPA Reference Documentation*. Retrieved June 2026, from `https://docs.spring.io/spring-data/jpa/reference/`.

[12] VMware Tanzu. *Spring Security Reference Documentation*. Retrieved June 2026, from `https://docs.spring.io/spring-security/reference/`.

[13] JetBrains. *Kotlin Documentation*. Retrieved June 2026, from `https://kotlinlang.org/docs/home.html`.

[14] Apache Software Foundation. *Apache Maven Documentation*. Retrieved June 2026, from `https://maven.apache.org/guides/`.

[15] Red Hat. *Hibernate ORM Documentation*. Retrieved June 2026, from `https://hibernate.org/orm/documentation/`.

[16] jwtk Contributors. *Java JWT (JJWT) Documentation*. Retrieved June 2026, from `https://github.com/jwtk/jjwt`.

[17] Liquibase Inc. *Liquibase Documentation: Changelogs*. Retrieved June 2026, from `https://docs.liquibase.com/concepts/changelogs/home.html`.

[18] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. Retrieved June 2026, from `https://www.postgresql.org/docs/`.

[19] SpringDoc. *springdoc-openapi Documentation*. Retrieved June 2026, from `https://springdoc.org/`.

[20] Twilio Inc. *Twilio Messaging API Documentation*. Retrieved June 2026, from `https://www.twilio.com/docs/messaging`.

[21] LibrePDF Contributors. *OpenPDF Documentation*. Retrieved June 2026, from `https://github.com/LibrePDF/OpenPDF`.

[22] FasterXML. *Jackson Documentation*. Retrieved June 2026, from `https://github.com/FasterXML/jackson-docs`.

[23] JUnit Team. *JUnit 5 User Guide*. Retrieved June 2026, from `https://junit.org/junit5/docs/current/user-guide/`.

[24] Kotest Contributors. *Kotest Documentation*. Retrieved June 2026, from `https://kotest.io/docs/framework/framework.html`.

[25] MockK Contributors. *MockK Documentation*. Retrieved June 2026, from `https://mockk.io/`.